

LECTURE 3

Java Threads & Concurrency

Concurrent Execution & Thread Management

Today's Agenda

- 1 Understanding Threads: Concepts & Terminology
- 2 Creating Threads: Extending vs Implementing
- 3 Thread Lifecycle & State Management
- 4 Synchronization & Race Conditions
- 5 Advanced Concurrency: Deadlock & Thread Pools
- 6 Best Practices & Performance Considerations

PART 1

Understanding Threads

The fundamentals of concurrent execution

What is a Thread?

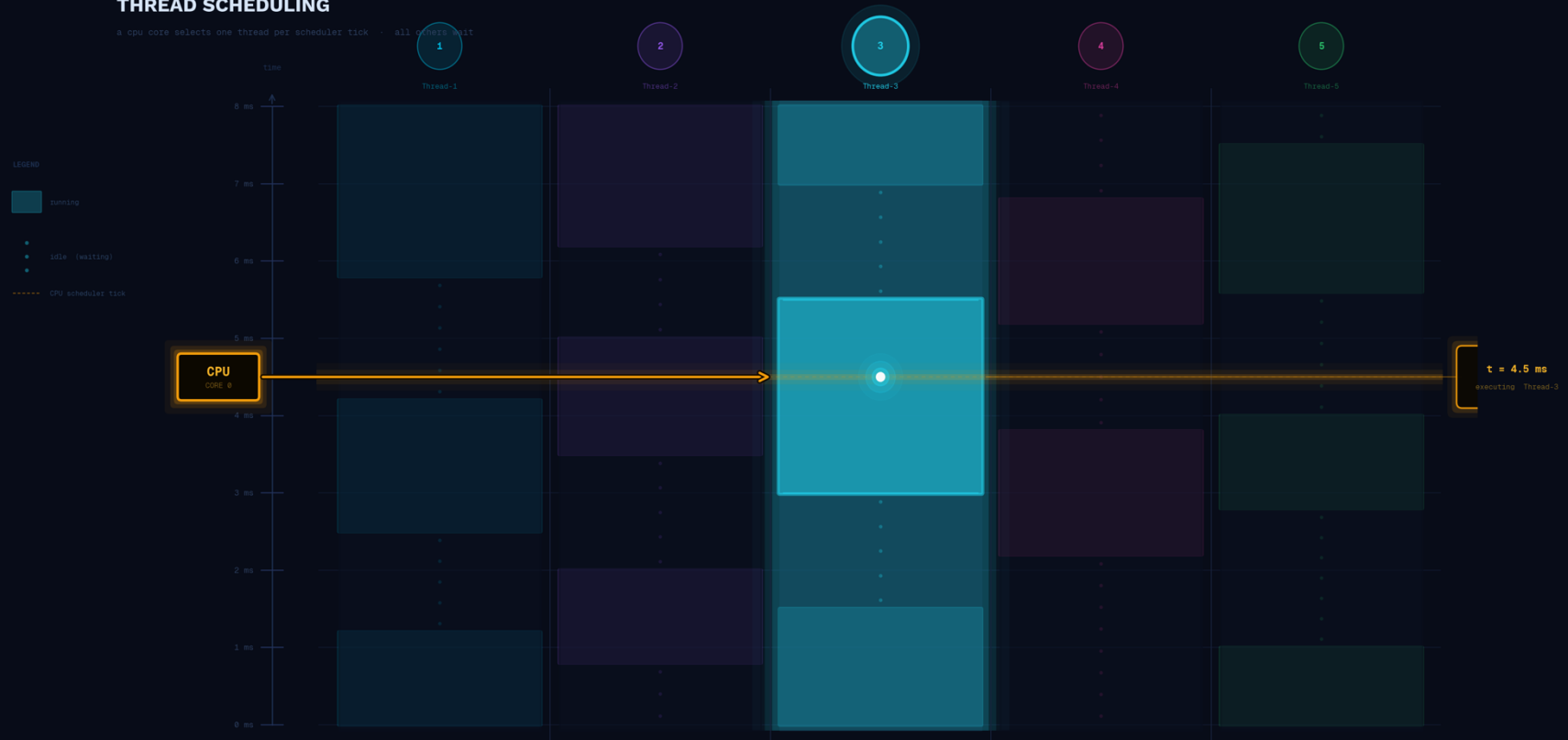
A thread is a lightweight unit of execution within a process. It represents an independent path of code execution that can run simultaneously with other threads.

- Each thread has its own stack and program counter
- Threads share the same memory space (heap) of their parent process
- Lighter weight than creating separate processes
- Threads can be created and destroyed more efficiently than processes

Lecture 3: Java Threads & Concurrency

THREAD SCHEDULING

a cpu core selects one thread per scheduler tick - all others wait



Process vs Thread

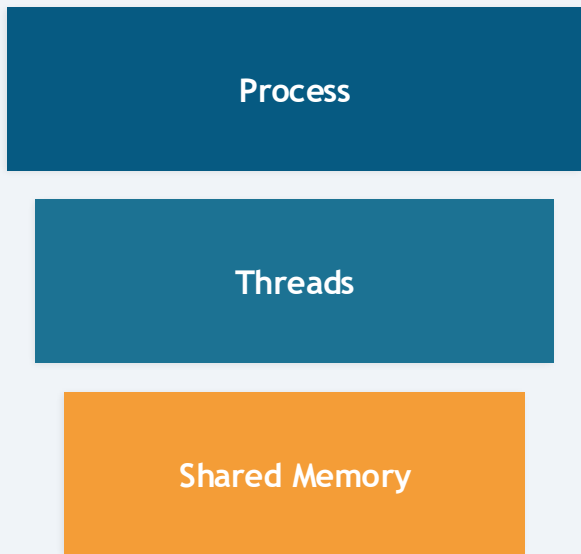
Process

- Independent program execution
- Separate memory space
- Higher overhead to create
- Takes longer to context switch
- More isolated and secure
- Examples: running Chrome, Firefox

Thread

- Execution within a process
- Shared memory space
- Lower overhead to create
- Faster context switching
- Direct access to process resources
- Multiple threads per application

Process vs Thread Structure



- Process contains one or more threads sharing heap memory
- Each thread has independent stack (local variables, call stack)
- Thread creation is lightweight (microseconds vs milliseconds)
- Context switching between threads is cheaper than processes
- Shared memory enables efficient data exchange between threads

Why Use Threads?

- Responsiveness: Keep UI responsive while performing background tasks (network, file I/O)
- Parallelism: Utilize multi-core processors to execute code simultaneously
- Resource Sharing: Threads share memory, making data exchange efficient
- Simplicity: Easier than managing multiple processes
- Asynchronous Operations: Handle multiple tasks concurrently without blocking
- Server Applications: Serve multiple clients simultaneously (web servers, chat applications)

Single-threaded vs Multi-threaded Execution

1

Single-threaded

Only one task executes at a time. If one task blocks (I/O), entire program waits.

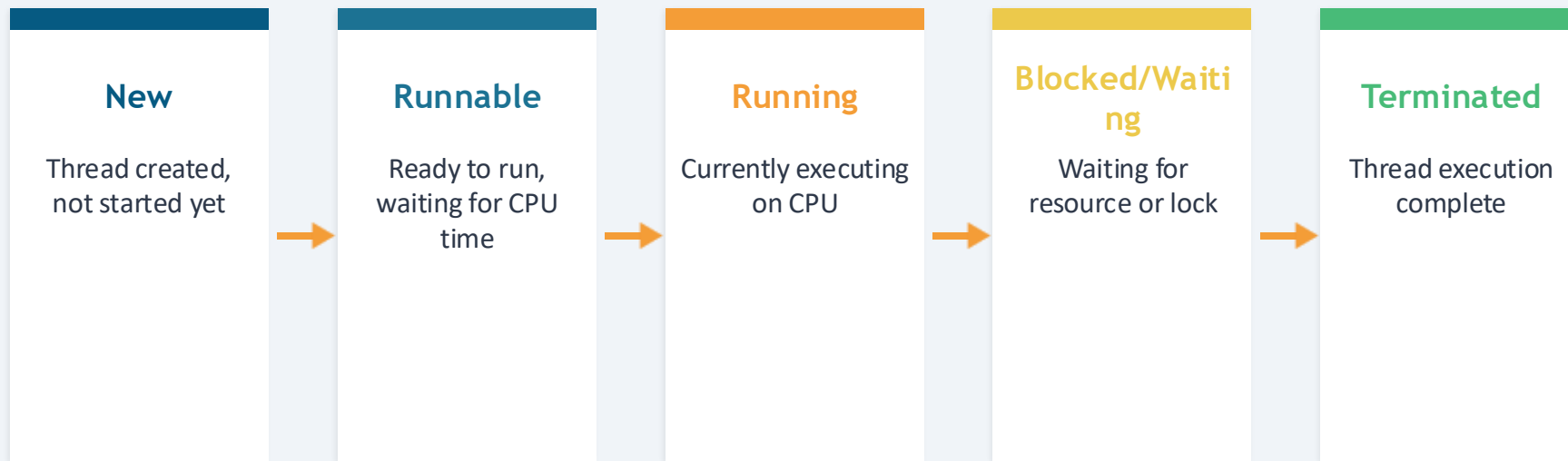
2

Multi-threaded

Multiple tasks execute concurrently. If one thread blocks, others continue running.

Thread Lifecycle States

How threads transition through different states



PART 2

Creating Threads

Two approaches: Extending Thread vs Implementing Runnable

Method 1: Extend Thread Class

- Create class extending `java.lang.Thread`
- Override `run()` method with thread logic
- Create instance and call `start()` method
- Simple approach but limits inheritance
- Thread state methods: `start()`, `run()`, `sleep()`, `join()`, `interrupt()`
- Avoid calling `run()` directly—always call `start()`

Extending Thread: Example

JAVA

```
class MyThread extends Thread {
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println("Thread: " + i);
            try {
                Thread.sleep(1000); // Sleep 1 second
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread thread = new MyThread();
        thread.start(); // NOT thread.run()
        System.out.println("Main continues");
    }
}
```

start() spawns new thread; run() executes in current thread

Method 2: Implement Runnable Interface

- Create class implementing `java.lang.Runnable`
- Implement `run()` method with thread logic
- Create Thread instance passing Runnable
- Preferred approach—allows inheritance from another class
- More flexible for thread pool frameworks (Executor, etc.)
- Modern Java prefers this approach

Implementing Runnable: Example

JAVA

```
class MyRunnable implements Runnable {
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println("Thread: " + i);
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Thread thread = new Thread(new MyRunnable());
        thread.start();
        System.out.println("Main continues");
    }
}
```

Runnable provides better design flexibility

Java 8 Lambda: Concise Runnable

JAVA

```
public class Main {  
    public static void main(String[] args) {  
        // Lambda implementation of Runnable  
        Thread thread = new Thread(() -> {  
            for (int i = 0; i < 5; i++) {  
                System.out.println("Thread: " + i);  
                try {  
                    Thread.sleep(1000);  
                } catch (InterruptedException e) {  
                    e.printStackTrace();  
                }  
            }  
        });  
  
        thread.start();  
        System.out.println("Main continues");  
    }  
}
```

Lambdas make concurrent code cleaner and more readable

Why Runnable is the Better Choice

① Keeps inheritance free

Java allows only single inheritance. Extending Thread locks you out of extending any other class. Runnable leaves the slot open.

② Separates task from thread

Runnable defines what to run; Thread decides how. Decoupling logic from the thread mechanism makes code cleaner and reusable.

③ Fits the Executor framework

ExecutorService, thread pools, and lambdas all accept Runnable. Thread subclasses do not fit this modern API.

extends Thread ✗ vs **implements Runnable** ✓

✗ extends Thread

```
// Blocks inheritance!  
class Worker extends Thread {  
    public void run() {  
        doWork();  
    }  
}
```

```
// Can't also extend:  
// class Worker extends Thread  
//     extends BaseService ✗
```

```
// Tied to Thread API  
Worker w = new Worker();  
w.start();
```

✓ implements Runnable

```
// Inheritance still free  
class Worker extends BaseService  
    implements Runnable {  
    public void run() {  
        doWork();  
    }  
}
```

```
// Works with any executor  
Runnable r = new Worker();  
new Thread(r).start();  
executor.submit(r); // ✓  
pool.execute(r);    // ✓
```

Rule of thumb: if you only need to run a task in a thread, implement **Runnable**. Only extend **Thread** if you need to customize thread behaviour itself.

PART 3

Thread Lifecycle & State Management

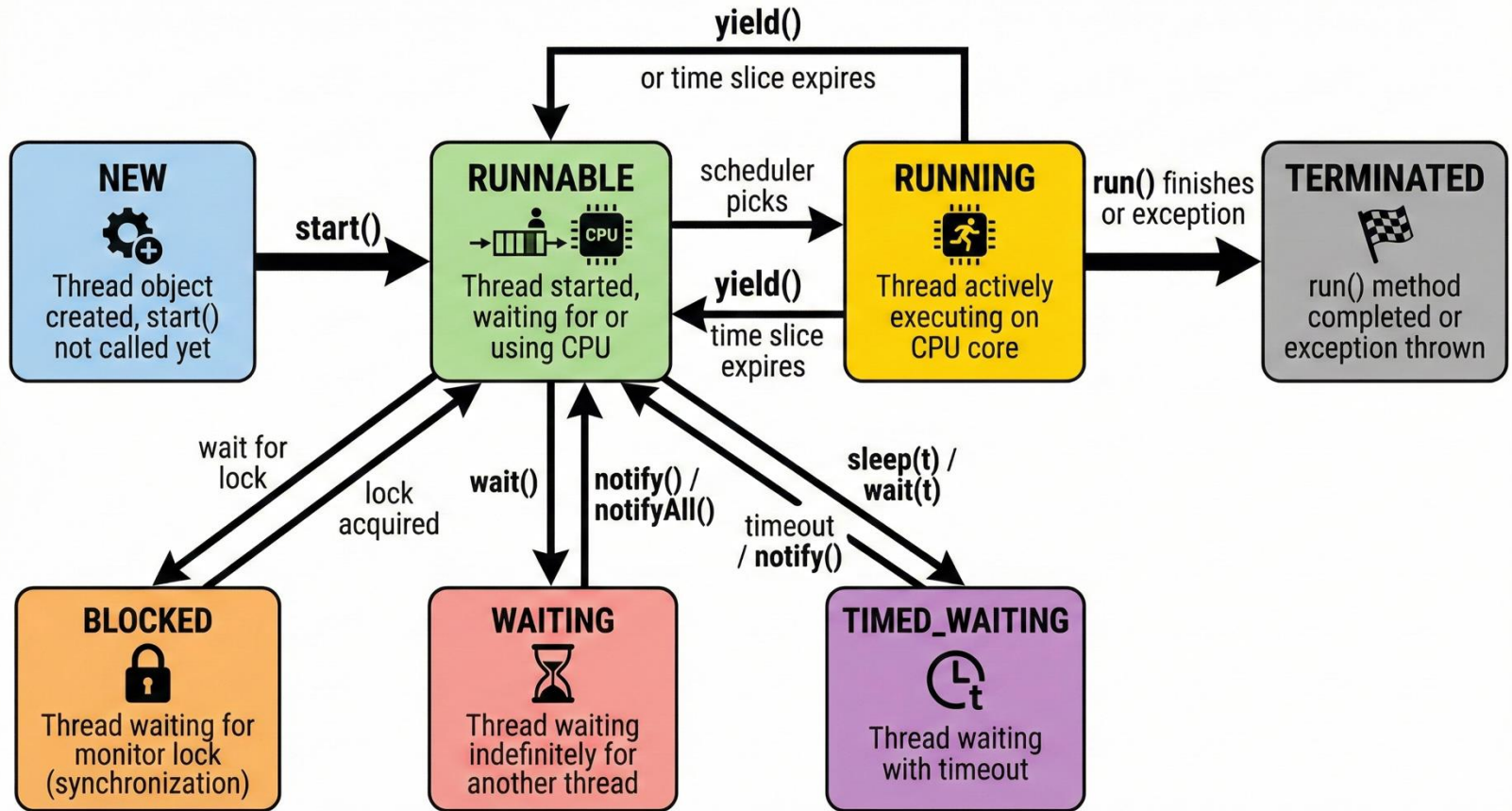
Managing thread execution states

Thread States Detailed

- NEW: Thread object created, start() not called yet
- RUNNABLE: Thread started, waiting for or using CPU
- RUNNING: Thread actively executing on CPU core
- BLOCKED: Thread waiting for monitor lock (synchronization)
- WAITING: Thread waiting indefinitely for another thread
- TIMED_WAITING: Thread waiting with timeout
- TERMINATED: run() method completed or exception thrown

Key Thread Methods

- `start()`: Begin thread execution (creates new call stack)
- `run()`: Contains thread logic (overridden by subclass/Runnable)
- `sleep(ms)`: Pause current thread for milliseconds
- `join()`: Wait for another thread to complete
- `interrupt()`: Send interrupt signal to thread
- `yield()`: Suggest to scheduler to switch to another thread
- `getId()`: Get unique thread identifier



Thread.join() Example

JAVA

```
public class Main {  
    public static void main(String[] args)  
        throws InterruptedException {  
        Thread thread1 = new Thread(() -> {  
            System.out.println("Thread 1 starting");  
            try { Thread.sleep(2000); }  
            catch (InterruptedException e) { }  
            System.out.println("Thread 1 done");  
        });  
  
        thread1.start();  
        System.out.println("Waiting for thread 1...");  
        thread1.join(); // Main waits for thread1  
        System.out.println("All threads complete");  
    }  
}
```

join() blocks current thread until target thread terminates

Without join() – Unpredictable Output

```
public class Main {  
    public static void main(String[] args)  
        throws InterruptedException {  
        Thread t1 = new Thread(() -> {  
            System.out.println("1: starting");  
            try { Thread.sleep(2000); }  
            catch (InterruptedException e) { }  
            System.out.println("1: done");  
        });  
  
        t1.start();  
        // NO join() here!  
        System.out.println("All threads complete");  
    }  
}
```

⚠ Possible Output (non-deterministic)

```
1: starting  
All threads complete  
1: done
```

— or —

```
All threads complete  
1: starting  
1: done
```

Why? Main thread doesn't block:

- * `t1.start()` fires `t1` in the background
- * Main immediately prints "done"
- * `t1` may not have run yet!

Output order is non-deterministic.

Without join(), main thread races ahead — output order is non-deterministic

PART 4

Synchronization & Race Conditions

Protecting shared data in concurrent environments

Race Condition

Situation where multiple threads access shared data simultaneously, causing unpredictable behavior and data corruption.

- Occurs when threads don't synchronize access to shared variables
- Timing and interleaving of thread execution affects outcome
- Hard to reproduce and debug (varies by hardware, JVM, load)
- Example: two threads incrementing same counter simultaneously

Concurrency Challenges

Race Condition

Multiple threads modify shared data without coordination. Result depends on timing.

Deadlock

Two threads wait for locks held by each other. Neither can proceed. System freezes.

Starvation

Thread never gets CPU time or lock. Lower priority threads starved by higher priority.

Livelock

Threads keep busy but make no progress. Like two people stepping same direction repeatedly.

Synchronization Mechanisms

- `synchronized` keyword: Ensures only one thread in critical section at a time
- `volatile` keyword: Ensures visibility of changes across threads
- Locks (`ReentrantLock`): More flexible than `synchronized`
- Atomic variables: Atomic operations without explicit locking
- `java.util.concurrent` utilities: `ConcurrentHashMap`, `BlockingQueue`, etc.
- Semaphore, `CountDownLatch`, `CyclicBarrier`: Advanced coordination

Without Synchronization vs With Synchronization

WITHOUT SYNCHRONIZATION

- Two threads increment shared counter simultaneously
- Thread A reads counter (value=100)
- Thread B reads counter (value=100)
- Thread A increments, writes back (value=101)
- Thread B increments, writes back (value=101)
- Expected 102, got 101 — one increment lost!



WITH SYNCHRONIZATION

- Two threads increment shared counter simultaneously
- Thread A acquires lock, reads counter (value=100)
- Thread B waits for lock...
- Thread A increments, writes back (value=101), releases lock
- Thread B acquires lock, reads counter (value=101)
- Thread B increments, writes back (value=102) — correct!

Thread Race — Non-Deterministic Output

```
class ThreadRace extends Thread {
    ThreadRace(String name) {
        super(name); // sets the thread's name
    }

    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(getName() + ": " + i);
        }
    }

    public static void main(String[] args) {
        new ThreadRace("A").start();
        new ThreadRace("B").start();
        new ThreadRace("C").start();
    } // threads race – no join!
}
```

Sample Output – getName() prints "A", "B", or "C"

Run 1	Run 2
A: 1	C: 1
B: 1	A: 1
A: 2	B: 1
C: 1	C: 2
B: 2	A: 2
...	...

Key Insight

The OS scheduler decides which thread runs next — your code has no control over ordering.

Same code, different run = different interleaving. Always.

This is the fundamental nature of threads.

getName() is inherited from Thread — no override needed; super(name) sets it at construction

Synchronized Method Example

JAVA

```
class BankAccount {  
    private int balance = 1000;  
  
    // Synchronize entire method  
    public synchronized void withdraw(int amount) {  
        if (balance >= amount) {  
            balance -= amount; // Critical section  
        }  
    }  
  
    public synchronized void deposit(int amount) {  
        balance += amount; // Critical section  
    }  
  
    public synchronized int getBalance() {  
        return balance;  
    }  
}
```

synchronized prevents concurrent access. Performance cost: lock overhead

Synchronized Block Example

JAVA

```
class BankAccount {  
    private int balance = 1000;  
  
    public void withdraw(int amount) {  
        // Only synchronize critical section  
        synchronized(this) {  
            if (balance >= amount) {  
                balance -= amount;  
            }  
        }  
        // Rest of method runs without lock  
        System.out.println("Withdrawal processed");  
    }  
  
    public void logTransaction(String msg) {  
        // This runs without lock  
        System.out.println(msg);  
    }  
}
```

Synchronized blocks are more fine-grained than synchronized methods

Concurrency in Real World: Ticket Booking

10 min

SCENARIO

You are building a ticket booking system for a concert. 50,000 users are trying to buy from 5,000 available tickets simultaneously. Two users click 'Buy' on the last ticket at the exact same millisecond.

THINK ABOUT

1

What happens without synchronization?

2

How would you ensure only one user gets the ticket?

3

What's the trade-off between safety and performance?

PART 5

Advanced Concurrency: Deadlock & Thread Pools

Handling complex concurrent scenarios

Deadlock: Definition and Conditions

- Deadlock: Situation where two or more threads wait for locks held by each other
- Four conditions must all be present:
 1. Mutual Exclusion: Resource cannot be shared
 2. Hold and Wait: Thread holds resource while waiting
 3. No Preemption: Locks cannot be forcibly taken
 4. Circular Wait: Cyclic chain of threads waiting

Deadlock Formation Example

Circular dependency of locks



Deadlock Example (What NOT to do)

JAVA

```
class DeadlockExample {
    static Object lock1 = new Object();
    static Object lock2 = new Object();

    static void threadA() {
        synchronized (lock1) {
            System.out.println("A: got lock1");
            try { Thread.sleep(100); }
            catch (InterruptedException e) {}
            synchronized (lock2) { // DEADLOCK risk!
                System.out.println("A: got lock2");
            }
        }
    }
}
```

```
static void threadB() {
    synchronized (lock2) {
        System.out.println("B: got lock2");
        try { Thread.sleep(100); }
        catch (InterruptedException e) {}
        synchronized (lock1) { // DEADLOCK!
            System.out.println("B: got lock1");
        }
    }
}

public static void main(String[] args) {
    Thread tA = new Thread(DeadlockExample::threadA);
    Thread tB = new Thread(DeadlockExample::threadB);
    tA.start(); tB.start();
}
}
```

Lecture 3: Java Threads & Concurrency

DEADLOCK

each thread holds one lock and waits for the other : neither can ever proceed



Preventing Deadlock

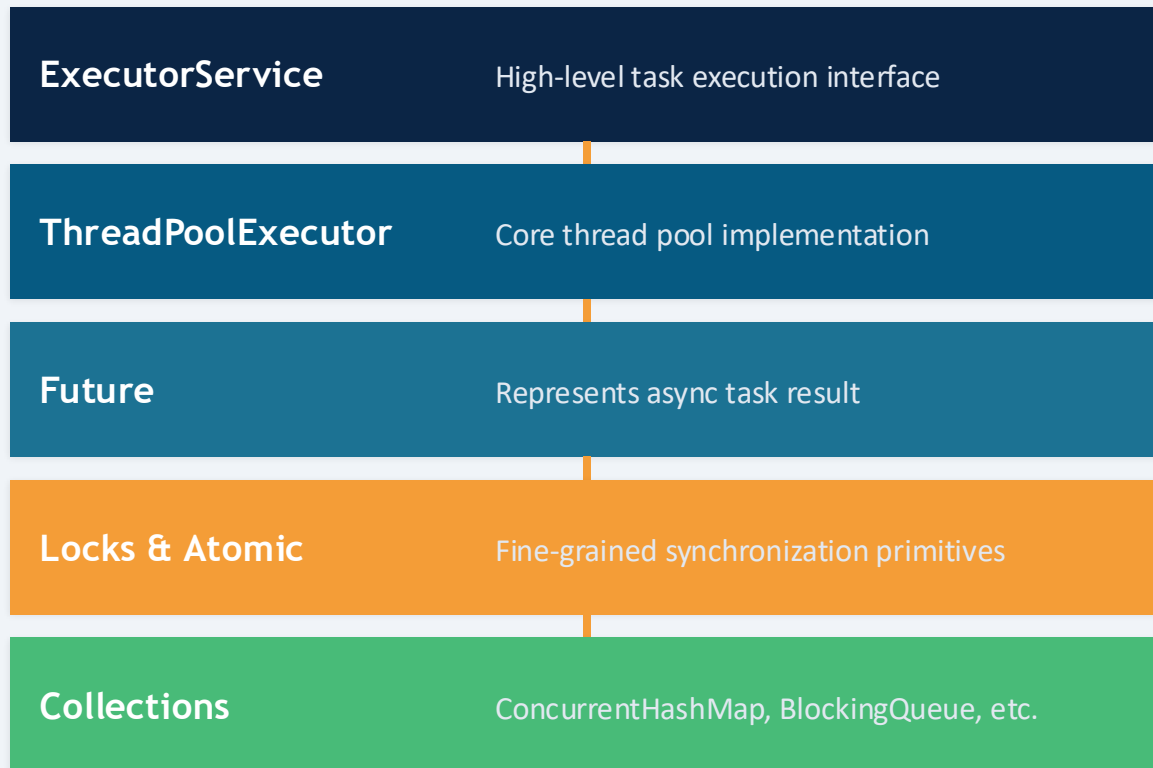
- Acquire locks in consistent order across all threads
- Hold locks for minimal duration
- Use timeout-based locks (tryLock with timeout)
- Avoid nested locks when possible

Thread Pool (Executor Framework)

Manages pool of reusable threads for executing tasks efficiently.

- Avoids creating new threads for each task (expensive)
- Reuses threads from pool for multiple tasks
- `java.util.concurrent.ExecutorService`: Main interface
- `ThreadPoolExecutor`: Implementation with configurable size

java.util.concurrent Components



Key Points

- High-level abstractions
- Preferred over manual synchronization
- Better scalability and performance

ExecutorService: Thread Pool Example

JAVA

```
ExecutorService executor = Executors.newFixedThreadPool(3);

for (int i = 0; i < 10; i++) {
    int taskNum = i;
    executor.execute(() -> {
        System.out.println("Task " + taskNum +
            " running on " +
            Thread.currentThread().getName());
        try { Thread.sleep(2000); }
        catch (InterruptedException e) { }
    });
}

executor.shutdown();
executor.awaitTermination(10, TimeUnit.SECONDS);
System.out.println("All tasks complete");
```

Pool of 3 threads handles 10 tasks. Much more efficient than 10 threads

Future: Async Task Result

JAVA

```
ExecutorService executor = Executors.newFixedThreadPool(2);

Future<Integer> future = executor.submit(() -> {
    System.out.println("Computing...");
    Thread.sleep(2000);
    return 42;
});

System.out.println("Main continues...");
try {
    Integer result = future.get(); // Block until done
    System.out.println("Result: " + result);
} catch (Exception e) {
    e.printStackTrace();
}

executor.shutdown();
```

future.get() blocks until result available. Use timeouts in production

Think — Pair — Share

5 min

Understanding data consistency in concurrent systems

A bank application has Account A with \$1000 and Account B with \$1000. Thread 1 transfers \$500 from A to B. Thread 2 transfers \$300 from B to A. Both run simultaneously. What could go wrong?

1 THINK

Reflect individually for 1 minute

2 PAIR

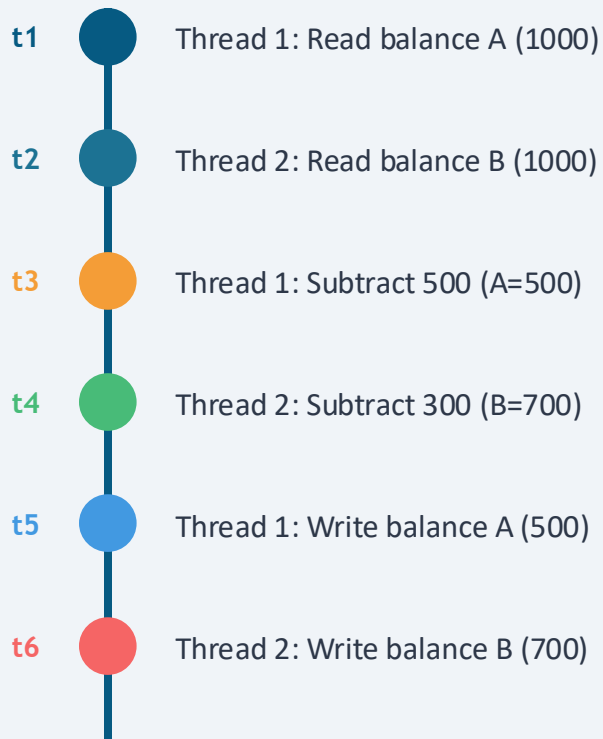
Discuss with your neighbor for 2 minutes

3 SHARE

Share insights with the class

Hints: Read-modify-write | Concurrent access | Consistency

Thread Execution: Interleaving Example



PART 6

Best Practices & Performance

Writing correct and efficient concurrent code

Concurrency Best Practices

- Prefer `java.util.concurrent` over manual synchronization
- Use immutable objects when possible (eliminates race conditions)
- Minimize lock scope and lock holding time
- Test concurrent code with multiple runs and stress tests

Immutable Objects for Thread Safety

JAVA

```
// Immutable = thread-safe automatically
final class ImmutablePoint {
    private final int x;
    private final int y;

    public ImmutablePoint(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; }
    public int getY() { return y; }

    // No setters!
}

// Multiple threads can safely share:
ImmutablePoint p = new ImmutablePoint(10, 20);
// Even with 1000 threads, no race condition!
```

Immutability is the simplest way to achieve thread safety

Common Concurrency Mistakes

- Using synchronized keyword ineffectively (too broad or too narrow)
- Calling run() instead of start() (executes in same thread)
- Creating threads without pools (1000s of threads = disaster)
- Not using thread-safe collections (Vector is old; use ArrayList + sync)

Key Concurrency Takeaways

- 1 Threads enable responsive, parallel execution but introduce complexity
- 2 Race conditions occur when threads access shared data without synchronization
- 3 synchronized keyword and locks prevent race conditions
- 4 Deadlock occurs with circular lock dependencies—prevent with lock ordering
- 5 Thread pools (ExecutorService) more efficient than manual thread creation
- 6 Prefer immutable objects and `java.util.concurrent` over manual synchronization

When to Use Concurrency

- UI applications: Keep event thread responsive with background threads
- Server applications: Handle multiple clients with thread pool
- Parallel processing: Divide work across CPU cores
- Asynchronous operations: Network/file I/O without blocking

Questions?

Next: Lecture 4 — Socket Programming & Network Protocols

CSCI 2450 | Computer Interface Programming | 2026